The Relay: an evidence-first RAG assistant over three papers

1 What I built and why

A Python assistant that answers only from three pinned arXiv papers (Attention v7, Moshi v1, Dynamic Chunking v2). Every answer carries a paper, a page, a section and a verbatim quote; the quote is checked by a tool call against the cited chunk *before* the answer is shown, and the system refuses rather than guess. Pipeline: analyse the question (is the corpus enough, which passages) -> BM25 retrieval over headings plus body (349 chunks, 350 tokens, 60 overlap) -> draft with JSON citations -> `verify_quote` -> at most two repairs -> refuse.

The decision that shaped everything else: **admission control on tokens, not retries on 429.** The free tier allows 30 requests per minute but only 8,000 tokens per minute. At roughly 1,200 tokens per answer the token ceiling binds about ten times earlier than the request ceiling, so the scheduler estimates cost before sending and refuses with a retry hint when a request cannot finish inside its deadline. Retries exist, but only for genuine failures.

2 The real service and my stand-in

Documented free-tier limits for `openai/gpt-oss-120b` (console.groq.com/docs/rate-limits, read 9 Nov 2026): 30 RPM, 1,000 RPD, 8,000 TPM, 200,000 TPD, 429 with `retry-after`. `relay probe-limits` is written to measure the observed side - 32 concurrent minimal calls, two large prompts for TPM, header capture, recovery polling - but **no key was in `.env` at submission time**, so Table 1's observed column says *not measured*. I did not dress documentation up as measurement.

The stand-in (`services/standin`) speaks the same OpenAI SSE protocol with real counters for those limits, so client code is identical for both sources. Its defaults are **uncalibrated**, and `/health` says `calibrated: false`; `relay calibrate` would fit them to real calls. Where it cannot match: no cold start, TTFT independent of prefill length, provider queue depth approximated as a linear slowdown. Every CSV row carries `source`, and every load number here is labelled stand-in.

Table 1 - limits: documented versus observed

Limit	Documented	Observed	How found	Confidence
Requests/min	30	not measured	probe not run (no key)	documented
Tokens/min	8,000	not measured	probe not run (no key)	documented
Requests/day	1,000	not measured	probe not run (no key)	documented
Tokens/day	200,000	not measured	would cost 200k tokens to reach	documented
Requests in flight	-	not measured	probe not run (no key)	not measured
On limit hit	429 + `retry-after`	not measured	probe not run (no key)	not measured
Time to recover	-	not measured	probe not run (no key)	not measured

3 Results

493 requests: 60 evaluation (20 questions x 3 settings) and 433 load, 158k tokens in, 10.5k out, 44 retries. Tables 2a/2b come from `results/results.csv` via `relay tables`. Latency covers **successful requests only**; refusals are counted in 2b, because a 7 ms refusal is not a 7 ms answer.

Table 2a - latency by scenario (ms), successful requests

Scenario	Source	Req	OK	Duration (s)	Offered/min	TTFT p50	TTFT p95	TTFT p99	Total p50	Total p95	Total p99
steady	stand-in	14	14	196.4	4	192	381	420	1,494	2,099	2,195
burst	stand-in	30	1	15.1	600	183	183	183	14,243	14,243	14,243
overload	stand-in	45	3	44.8	60	270	352	359	1,779	11,623	12,498
sustained 4 min	stand-in	240	30	239.0	60	188	753	1,029	1,205	1,681	1,905
faulty	stand-in	40	9	80.3	30	187	734	968	1,574	3,157	3,602
degraded	stand-in	30	12	175.0	12	210	439	556	1,200	14,546	23,381
limit halved	stand-in	22	10	90.3	25	225	751	903	17,146	45,491	49,427
daily exhausted	stand-in	12	0	11.0	60	not measured	not measured	not measured	not measured	not measured	not measured

Table 2b - reliability, retries, queue and cost

Scenario	Succeeded	Rate-limited	Timed out or given up	Retries/req	Peak queue	Tokens out/answer	Cost/1,000
steady	100.0%	0.0%	0.0%	0.00	1	75	$0.00
burst	3.3%	96.7%	0.0%	0.27	1	78	$0.00
overload	6.7%	93.3%	0.0%	0.18	1	82	$0.00
sustained 4 min	12.5%	87.5%	0.0%	0.03	1	66	$0.00
faulty	22.5%	62.5%	15.0%	0.20	1	66	$0.00
degraded	40.0%	43.3%	16.7%	0.27	1	66	$0.00
limit halved	45.5%	36.4%	18.2%	0.36	1	67	$0.00
daily exhausted	0.0%	100.0%	0.0%	0.00	1	not measured	$0.00

The number I could not have guessed: **30 requests per minute is not 30 answers per minute.** At 4/min everything succeeds (14/14, total p50 1.5 s). Held at 60/min for four minutes, only 12.5% succeed, and the per-minute refusal is instant. The advertised ceiling and the usable ceiling differ by roughly ten, and the difference is entirely tokens.

4 Where the time goes

Steady state: 1,494 ms p50, of which 192 ms to the first token (12.9%) and 1,299 ms after it (87.1%), with 0 ms queue wait and 0 ms retrying.

One slow request, chosen from the CSV by `max(total_ms) where status=ok`: **`2ff56ecb7f73`** (limit-halved, 31,832 ms). Its trace splits it as **limit wait 27,261 ms (85.6%) + working 1,050 ms (3.3%) + queue 0 + retrying 0**, with the first token at 392 ms. The delay is *before* the first token, and it is budget waiting: the limiter held the request until the token window freed, then the draft call took 1.05 s. It was not model slowness, not retries, not queueing.

That split did not exist until this was investigated: the earlier trace showed 50,411 ms total with 0 ms queued and 961 ms working, i.e. 49 s unaccounted. The gap was time blocked inside `limiter.acquire`, which nothing was timing. `limit_wait_ms` now records it, and the request above was re-run with that instrumentation. The split is structural: the first call decides answerability and is never shown, so the token a person sees is the second call's, and everything after it is streaming ~70 tokens plus verification.

5 When a limit is hit

Per-minute: the provider returns 429 with `retry-after`. If the wait fits the deadline the limiter sleeps and retries (at most twice); otherwise it refuses at once, quoting the wait. Halving both windows is the only scenario where waiting was rational: 45.5% succeeded, 36.4% refused, 18.2% waited and timed out at p95 45.5 s. Per-day: no wait inside a request restores a daily budget, so it rejects before sending - 0 retries, 0 ms, 100% `rate_limited`.

Degradation is now measured, not asserted. With 65% of the minute already spent, the ladder engages by itself: levels 0/1/2 appear as 22/21/17 of 30 requests, average context falls from 578 tokens (steady) to 411, and average model calls from 1.64 to 1.37 - it skips the analysis call, then trims context to one or two passages and drops the repair loop. In the halved run 16/22 requests ran at level 2. Nobody in these runs received a *weaker answer* rather than a refusal, because the bucket empties before the trimmed path can complete; what is measured is that the system sheds work before it sheds correctness.

Retrieval: quality versus cost

Table 3 - retrieval settings (stand-in source, automatic grades)

Setting	Piece	Fetched	Re-rank	Q	Correct+cited	Wrong	Refused correctly	Refused when answerable	Quotes verified	Tokens out	TTFT p50	TTFT p95	Total p50	Total p95
A	350/60	4	none	20	25.0%	75.0%	0.0%	0.0%	100.0%	81	167	591	1,680	2,368
B	700/100	4	none	20	20.0%	80.0%	0.0%	0.0%	100.0%	80	168	585	1,647	2,455
C	350/60	16	cross-encoder	20	25.0%	75.0%	0.0%	0.0%	100.0%	81	221	824	1,723	2,505

Larger chunks (B) buy nothing: same tokens, one fewer correct citation. Re-ranking (C) costs 54 ms of p50 TTFT and 233 ms at p95 for the same accuracy. Citation correctness is a separate axis, not folded into the grade: every quote the system shipped was matched exactly by `verify_quote` (100%), while *location* is what the gold-region grade tests. The 0% refusal rate is a property of the *stand-in*, which sets `answerable = bool(question)`; of the 21 unanswerable results all 21 came back answered, so refusal correctness here is 0/21 and a real model is what would change it (`docs/ANALYSIS.md` §1). Table 3 can therefore choose on cost only, and both B and C cost more than A. When the limit is close, retrieval is the first thing squeezed: the ladder cuts context to one passage (328 tokens in `2ff56ecb7f73`), so a near-limit answer is cheaper and shallower.

How this library will mislead a naive retriever

1. **Appendix gravity.** Moshi is 66 pages and supplies 172 of 349 chunks; appendices repeat "LibriSpeech" and "WER" densely. `q06` (streaming WER, section 5.7) returns appendix tables first - the citation looks right and points at the wrong section.
2. **Cross-paper term collision.** `q11` asks for batch size and training steps (Attention section 5.1); the top four fill with Moshi and H-Net training sections that use exactly those words. This is why gold is a page-and-section region, not a chunk id.
3. **Heading versus body.** `q01` asks for the key dimension in section 3.2, but the body repeats "attention" and "heads" everywhere, so a body-only index returns generic 3.2.1 text. Indexing headings with each chunk lifted that passage into the top four.

6 Retries

Full-jitter exponential backoff (base 400 ms, cap 8 s), three attempts per call, a shared retry budget, and a rule that the delay is never shorter than the provider's `retry-after`. The breaker counts only 5xx and timeouts: a 429 means the provider is healthy and busy.

Measured: 44 retries across 493 requests (0.09 each), 0.36 when limits are halved. Of 38 retried rows, 3 succeeded. **A controlled experiment where retrying is worse** (scripted provider, so it is reproducible in milliseconds and says nothing about Groq's real fault rate): one 500 with a 40 ms attempt, `max_attempts=1` fails in 1 attempt; `max_attempts=3` fails identically in 3 attempts and takes more than 2.5x the wall clock, all of it backoff. Same outcome, later. A 429 whose `retry-after` (55 s) exceeds the 30 s deadline is refused in under 0.5 s rather than slept on, which is the same lesson applied.

That policy came from a failure: the first evaluation counted 429s as breaker failures, opened the circuit for 20 s and turned 13 requests into instant `error` rows. The fix is decision D6 in `docs/DECISIONS.md`.

7 What the person sees

When the daily budget is gone, this is the reply, verbatim:

> *I can't answer this one right now: today's free-tier model budget is spent. The account allows 200,000 tokens and 1,000 requests per day, shared with every other run, and it resets at the start of the next UTC day. No partial or guessed answer was sent. Retry after the reset, or narrow the question so it needs fewer passages.*

At the per-minute limit the message names the wait ("It frees up in about 55 seconds. Nothing was lost - retry then.") and repeats the promise: no shorter, unverified answer was sent to stay fast.

8 What I am unsure about

Least certain: that `retry-after` estimates when capacity returns. It is a hint from one window; if it underestimates, the limiter sleeps, retries and fails anyway - strictly worse than refusing, and exactly what the 18.2% timeout share in the halved run looks like. What breaks first: multi-paper synthesis, where nothing forces a cross-paper answer to cite more than one paper. Also unresolved: the stand-in is uncalibrated, so its fault rates (15% 500s, 5% timeouts, 10% truncated streams in `faulty`) are synthetic and only prove the handling, not the frequency. Not built: multi-provider failover and an answer cache; neither changes the token ceiling. What I learned: a latency budget is set by the slowest *mandatory* step, and measurement can hide the biggest component - 27 s of a 32 s request was invisible until the limiter was instrumented.

Your feedback on Vaani

Checked against the live pages on 10 Nov 2026, not from memory.

Standout: "Zero dead air" and the screening section - Vaani answers the carrier prompt and stays on the line, with ~98% of screening calls handled under 15 s - are framed as outcomes, not model features. That is the right way to sell a voice agent, and it is the unglamorous work that decides whether outbound campaigns survive.

Confusing: the hero's three counters render as `0M+`, `0.0%` and `<0ms` with labels ("Response accuracy", "Sub-500ms... P-90 latency") that do not sit under the numbers they describe, so the most prominent evidence on the page reads as broken. The body separately claims "p90 latency of ~450ms", which is credible but unsourced. And the CTA says "Start free on Vaani's Platform" while `/pricing` offers only pay-as-you-go (prepaid wallet, ₹1/min platform fee, 2 concurrent calls) and Enterprise - so "free" is not a plan a reader can find.

One change: publish the measurement method beside the latency claim (audio conditions, line type, sample size) and make the hero counters carry their denominators. For a latency-led product the method is part of the pitch. *(Web pages only; I have not run a live call or spoken to the team, so the accuracy counter's provenance stays unverified.)*

How I worked with the assistant

Three prompts that mattered: "plan before coding, with a gate per stage"; "measure the limits before designing the limiter"; and "every table must regenerate from `results.csv`". One thing it got wrong: grading evaluation rows after the pipeline had written them, producing duplicate, partly ungraded rows - caught by counting rows against questions. Two things it got wrong later: the breaker treating 429s as outages (13 instant errors), and a 49 s unaccounted gap in a 50 s request that only appeared when one request was split instead of summarised. One overrule: it proposed a vector store for 349 chunks; BM25 over headings stayed. What is distinctly mine: the token-admission decision, verify-before-reveal, labelling every stand-in row, and this report's refusal to fill unmeasured cells.